

Dokumentacja projektu aplikacji dziennika elektronicznego dla szkół podstawowych oraz średnich

Jakub Nadolski

Igor Gula

12 czerwca 2026

Spis treści

1	Opis problemu	3
1.1	Opis i cel projektu	3
1.2	Porównanie dostępnych rozwiązań	3
1.3	Możliwości zastosowania praktycznego	3
2	Projekt i analiza	4
2.1	Wymagania funkcjonalne	4
2.2	Wymagania нефunkcjonalne	5
2.3	Diagram przypadków użycia	6
2.4	Diagramy klas	8
2.5	Diagram ERD dla relacyjnej bazy danych	9
2.6	Projekt interfejsu użytkownika	10
3	Implementacja	14
3.1	Architektura całości systemu	14
3.2	Użyte wzorce projektowe	14
3.3	Diagramy sekwencji przedstawiające funkcjonalności systemu	15
3.4	Użyte technologie w projekcie systemu	18
4	Testy	19
4.1	Analiza testów części frontendowej aplikacji	19
4.1.1	Zakres testów	19
4.1.2	Kluczowe przypadki testowe	21
4.1.3	Podsumowanie wyników testów	22
4.2	Analiza testów części backendowej aplikacji	23
4.2.1	Zakres testów	23
4.2.2	Kluczowe przypadki testowe	24
4.2.3	Podsumowanie wyników testów	25
4.3	Wnioski z testów	25

1 Opis problemu

1.1 Opis i cel projektu

Aplikacja to dziennik elektroniczny przeznaczony dla szkół podstawowych oraz średnich. Umożliwia rejestrację szkół, zarządzanie ich danymi oraz kontami użytkowników, klasami nauczycielami, uczniami, planami zajęć, ocenami i wnioskami rejestracji szkół. System wspiera różne role użytkowników (administrator aplikacji, administrator szkoły, nauczyciel, uczeń, rodzic) i przypisuje im odpowiednie uprawnienia, co pozwala kontrolować ich dostęp do operacji takich jak przeglądanie danych, wprowadzanie ich do systemu i zarządzanie użytkownikami. Od strony technicznej aplikacja składa się z warstwy frontendowej (TypeScript, React, Next.js), backendu (Java, SpringBoot) i bazy danych (PostgreSQL). Cel projektu to dostarczenie funkcjonalnego i skalowalnego rozwiązania do zarządzania pracą szkoły i komunikacją między użytkownikami.

1.2 Porównanie dostępnych rozwiązań

Na rynku jest dostępnych wiele systemów służących jako dzienniki elektroniczne w szkołach. Najbardziej rozpowszechnionymi z nich są komercyjne dzienniki elektroniczne takie jak Librus Synergia czy Vulcan UONET+, które umożliwiają prowadzenie dokumentacji szkolnej, zarządzanie ocenami, frekwencją oraz komunikacją między nauczycielami, rodzicami i uczniami. Oferują one szeroki zakres funkcjonalności, jednak są to zamknięte systemy, których rozwój i możliwości dostosowania do indywidualnych potrzeb użytkownika są ograniczone. Alternatywą mogą być systemy open-source. Ich zaletą jest możliwość modyfikacji kodu źródłowego i brak kosztów licencyjnych, jednak wdrożenie tych rozwiązań może wymagać zaawansowanej konfiguracji oraz dostosowania do lokalnych wymagań organizacyjnych. W ramach tego projektu zdecydowano się na opracowanie własnego systemu dziennika elektronicznego. Umożliwiło to pełną kontrolę nad funkcjonalnościami, elastyczność w rozwoju i architekturą aplikacji. Wybór technologii zapewnia dobre warunki do stworzenia nowoczesnych i responsywnych interfejsów, stabilnego API i przechowywania danych relacyjnych.

1.3 Możliwości zastosowania praktycznego

Opracowany system może znaleźć zastosowanie jako platforma wspomagająca zarządzanie informacjami w szkołach. Umożliwia on centralizację danych uczniów, nauczycieli, klas i ocen co sprzyja organizacji pracy szkoły i ogranicza konieczność prowadzenia dokumentacji w formie papierowej. Nauczyciele mogą używać systemu do wprowadzania i przeglądania ocen i frekwencji, rodzice i uczniowie do monitorowania postępów w nauce, a administratorzy do zarządzania strukturą organizacyjną szkoły i kontami użytkowników.

Architektura klient-serwer sprawia, że aplikacja może być dostępna z poziomu przeglądarki bez potrzeby instalowania dodatkowego oprogramowania. Ze względu na modułową budowę rozwiązanie może zostać łatwo rozbudowane o kolejne funkcjonalności zgodne z obecnymi potrzebami placówek edukacyjnych.

2 Projekt i analiza

2.1 Wymagania funkcjonalne

Wymagania funkcjonalne zostały podzielone według ról użytkowników, które są dostępne w systemie dziennika elektronicznego.

Niezaalogowany użytkownik

- Składanie wniosków o rejestrację szkoły w systemie.
- Logowanie się na konto.

Administrator aplikacji

- Zarządzanie danymi własnego konta (przeglądanie, edycja).
- Obsługa wniosków o rejestrację szkoły w systemie.
- Zarządzanie szkołami w systemie (tworzenie, przeglądanie, usuwanie).

Administrator szkoły

- Zarządzanie danymi własnego konta (przeglądanie, edycja).
- Zarządzanie danymi szkoły (przeglądanie, edycja).
- Zarządzanie członkami szkoły (tworzenie, przeglądanie, edycja, usuwanie).
- Zarządzanie klasami szkolnymi (tworzenie, przeglądanie, edycja, usuwanie).
- Zarządzanie przedmiotami szkolnymi (tworzenie, przeglądanie, edycja, usuwanie).
- Zarządzanie planami zajęć (tworzenie, przeglądanie, edycja, usuwanie).

Nauczyciel

- Zarządzanie danymi własnego konta (przeglądanie, edycja).
- Zarządzanie ocenami uczniów (tworzenie, przeglądanie, edycja, usuwanie).
- Zarządzanie frekwencjami uczniów (tworzenie, przeglądanie, edycja).
- Przeglądanie własnego planu zajęć.
- Przeglądanie danych o szkole.

Uczeń

- Zarządzanie danymi własnego konta (przeglądanie, edycja).
- Przeglądanie własnych ocen.
- Przeglądanie własnej frekwencji.
- Przeglądanie własnego planu zajęć.
- Przeglądanie danych o szkole.

Rodzic

- Zarządzanie danymi własnego konta (przeglądanie, edycja).
- Przeglądanie ocen swoich dzieci (uczniów).
- Przeglądanie frekwencji swoich dzieci (uczniów).
- Przeglądanie planów zajęć swoich dzieci (uczniów).
- Przeglądanie danych o szkole.

2.2 Wymagania нефункционалне

- System musi obsługiwać do 5000 jednocześnie zalogowanych użytkowników bez odczuwalnego opóźnienia w ładowaniu stron, które nie powinno przekraczać 2 sekund.
- Dziennik musi być dostępny dla użytkowników przez 99% czasu w ujęciu rocznym, wyłączając zaplanowane okna serwisowe ogłaszane z wyprzedzeniem.
- Wszystkie dane przesyłane między użytkownikiem a serwerem muszą być szyfrowane za pomocą protokołu TLS.

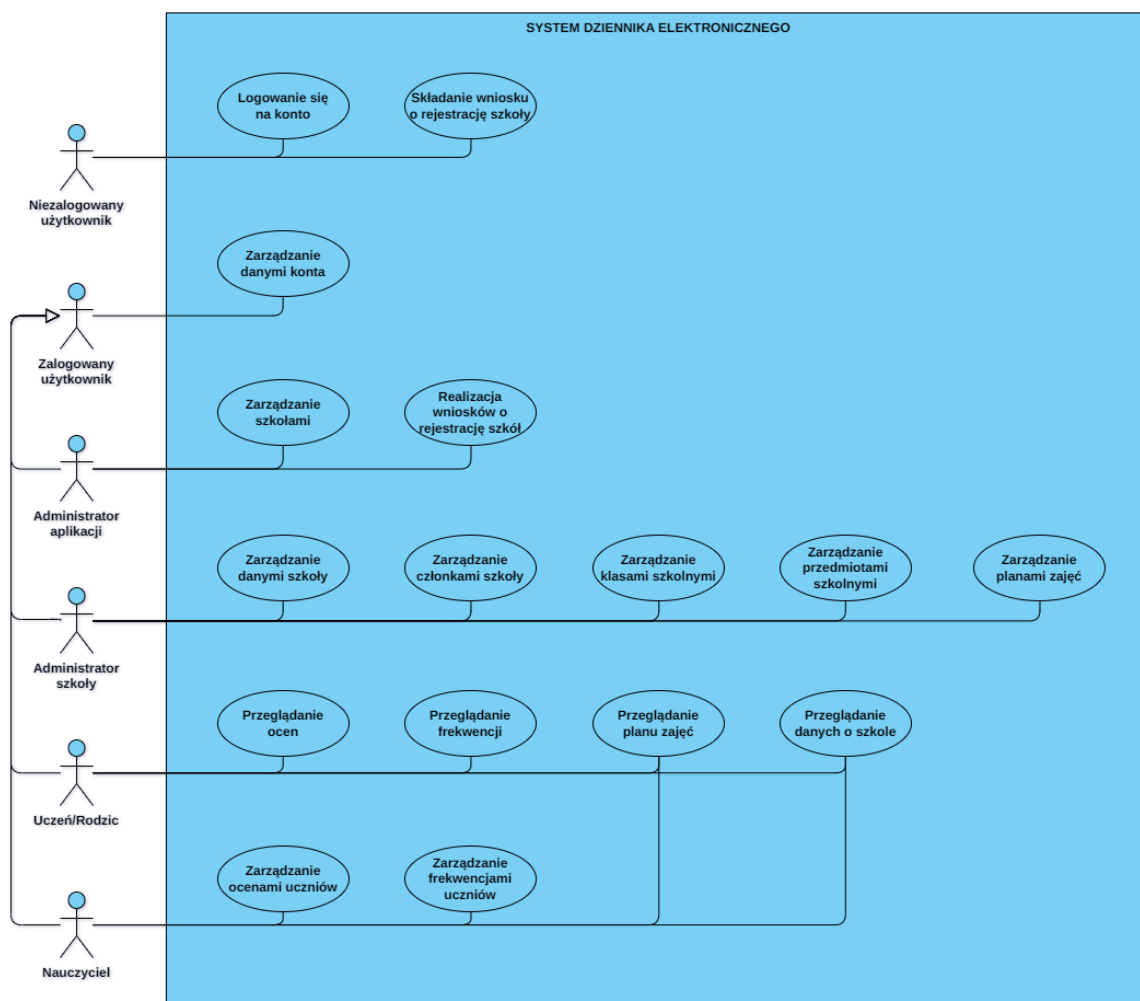
- System musi w pełni spełniać wymagania rozporządzenia RODO oraz aktualnych przepisów Ministerstwa Edukacji Narodowej dotyczących prowadzenia dokumentacji szkolnej.
- W przypadku awarii serwera głównego, system musi automatycznie przełączyć się na serwer zapasowy w czasie krótszym niż 5 minut bez utraty danych.
- Architektura systemu musi pozwalać na łatwe zwiększenie zasobów bazodanowych o 50% w okresie wystawiania ocen śródrocznych i końcowych bez konieczności przebudowy aplikacji.
- Aplikacja kliencka musi działać poprawnie na trzech najnowszych wersjach przeglądarek Google Chrome, Mozilla Firefox, Apple Safari i Microsoft Edge oraz posiadać responsywny interfejs dla urządzeń mobilnych.
- Każda zmiana oceny, frekwencji lub danych osobowych musi być trwale rejestrowana w dzienniku zdarzeń (logach) wraz z identyfikatorem autora i dokładnym czasem operacji.
- Pełna kopia zapasowa bazy danych musi być wykonywana automatycznie co 24 godziny i przechowywana w bezpiecznej, zewnętrznej lokalizacji przez okres minimum 5 lat.

2.3 Diagram przypadków użycia

Poniższy diagram przedstawia przypadki użycia systemu dziennika elektronicznego.

Na diagramie zostali przedstawieni aktorzy w systemie oraz funkcjonalności dostępne dla każdego aktora. Aktorzy ***Administrator aplikacji***, ***Administrator szkoły***, ***Uczeń***, ***Rodzic*** i ***Nauczyciel*** dziedziczą przypadki użycia po aktorze ***Zalogowany użytkownik***.

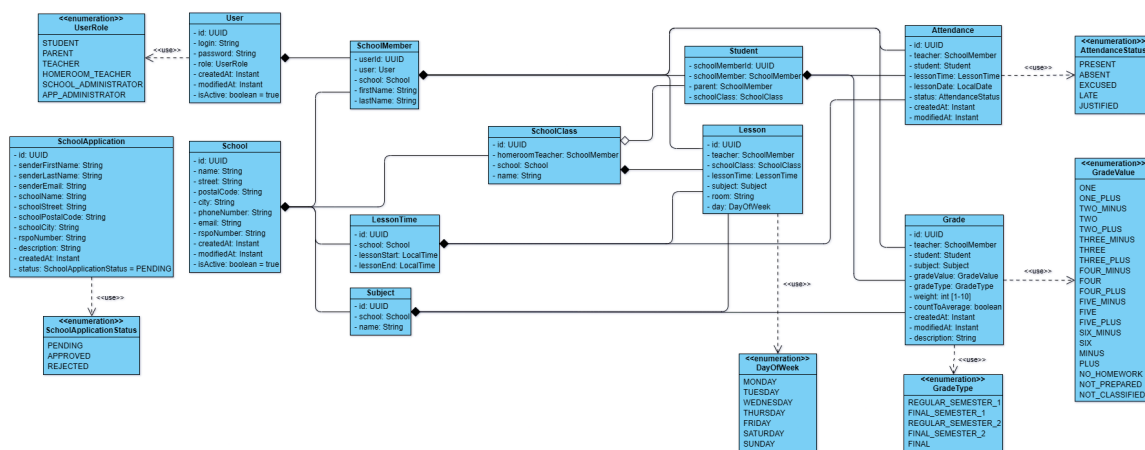
Przypadki użycia, którym nazwy zaczynają się od "***Zarządzanie...***"uwzględniają kilka lub wszystkie operacje CRUD. Szczegóły tych przypadków użycia zostały wypisane w sekcji wymagań funkcjonalnych.



Rysunek 1: Diagram przypadków użycia systemu dziennika elektronicznego.

2.4 Diagramy klas

Poniższy diagram przedstawia powiązania pomiędzy klasami encji bazodanowych w systemie dziennika elektronicznego.

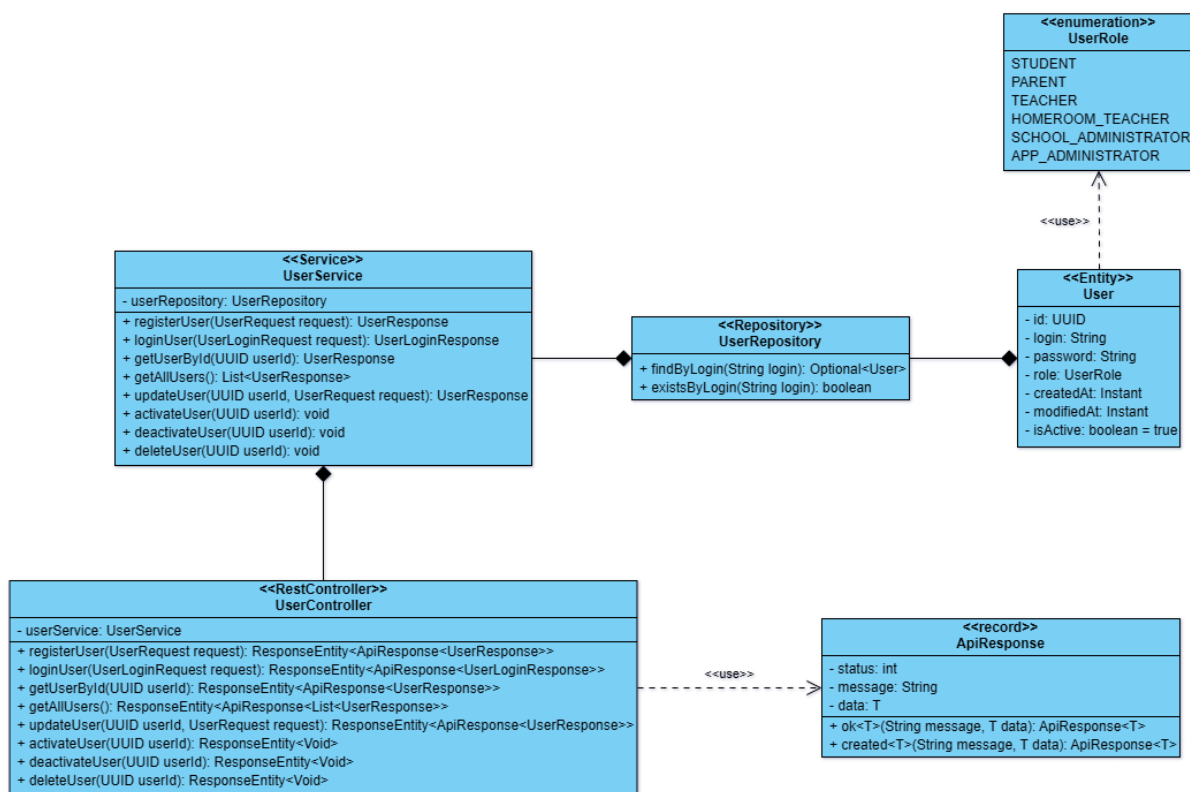


Rysunek 2: Struktura klas encji bazodanowych.

Aplikacja backedowa zajmująca się logiką biznesową stosuje wyraźny podział klas na:

- **Kontrolery** (*<nazwa_encji>Controller*) - zajmują się obsługą wystawianego interfejsu REST API.
- **Serwisy** (*<nazwa_encji>Service*) - zajmują się przetwarzaniem przekazanych danych i realizowaniem logiki biznesowej.
- **Repozytoria** (*<nazwa_encji>Repository*) - zajmują się komunikacją z bazą danych i wykonywaniem operacji na encjach bazodanowych.

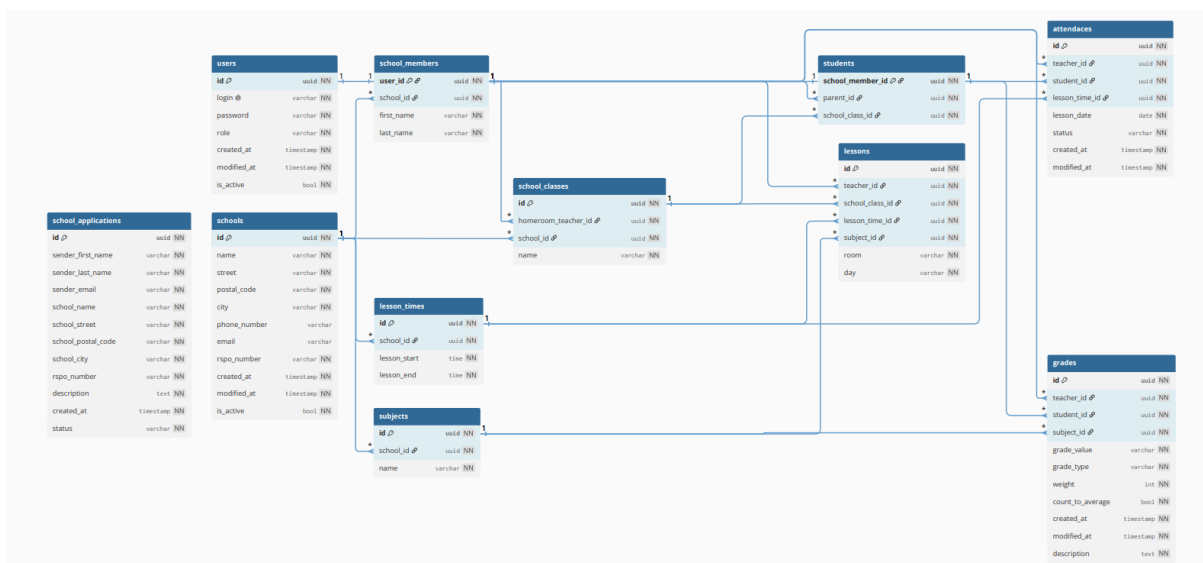
Poniższy diagram klas przedstawia powiązania klas dla encji bazodanowej **User**. Każda inna encja przedstawiona na pierwszym diagramie klas zachowuje podobną strukturę powiązań pomiędzy encjami, repozytoriami, serwisami i kontrolerami.



Rysunek 3: Powiązania klas dla encji User.

2.5 Diagram ERD dla relacyjnej bazy danych

Poniższy diagram przedstawia strukturę danych, która znajduje się w bazie danych systemu dziennika elektronicznego.



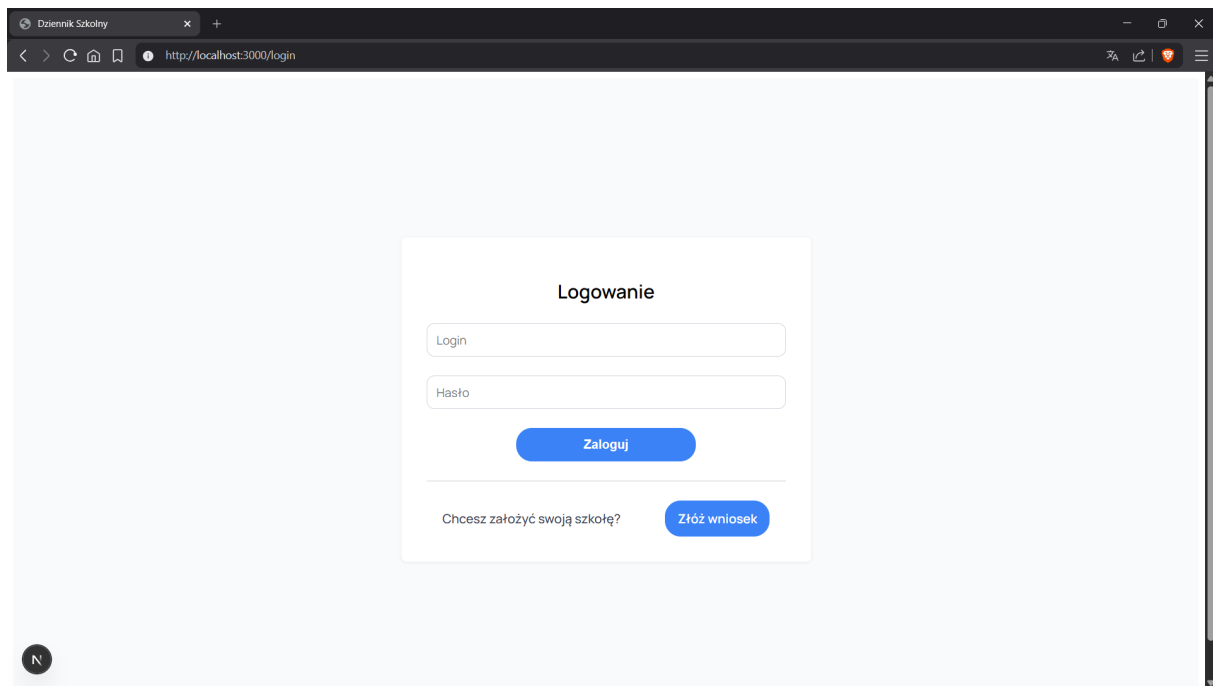
Rysunek 4: Diagram związków encji w bazie danych systemu.

Wyjaśnienie symboli:

- **NN** (Not Null) - oznacza, że kolumna tabeli nie może mieć pustej wartości (NULL).
- **Klucz** - oznacza, że kolumna tabeli jest kluczem głównym (identyfikatorem).
- **Łańcuch** - oznacza, że kolumna tabeli jest w relacji z inną tabelą.
- **Odcisk palca** - oznacza, że w danej kolumnie tabeli wszystkie wartości są unikalne (występuje tylko przy kolumnie *login* w tabeli *users*).

2.6 Projekt interfejsu użytkownika

Poniższe grafiki są gotowymi projektami interfejsu użytkownika, które są już zaimplementowane w aplikacji.



Rysunek 5: Strona logowania.

Wypełnij wniosek o rejestrację szkoły

Dane wnioskodawcy

Imię Nazwisko Email

Dane szkoły

Nazwa szkoły Numer RSPO

Adres szkoły

Miasto Kod pocztowy Ulica

Opis

[Wróć do logowania](#) [Zatwierdź](#)

Rysunek 6: Strona wniosku o rejestrację szkoły w systemie.

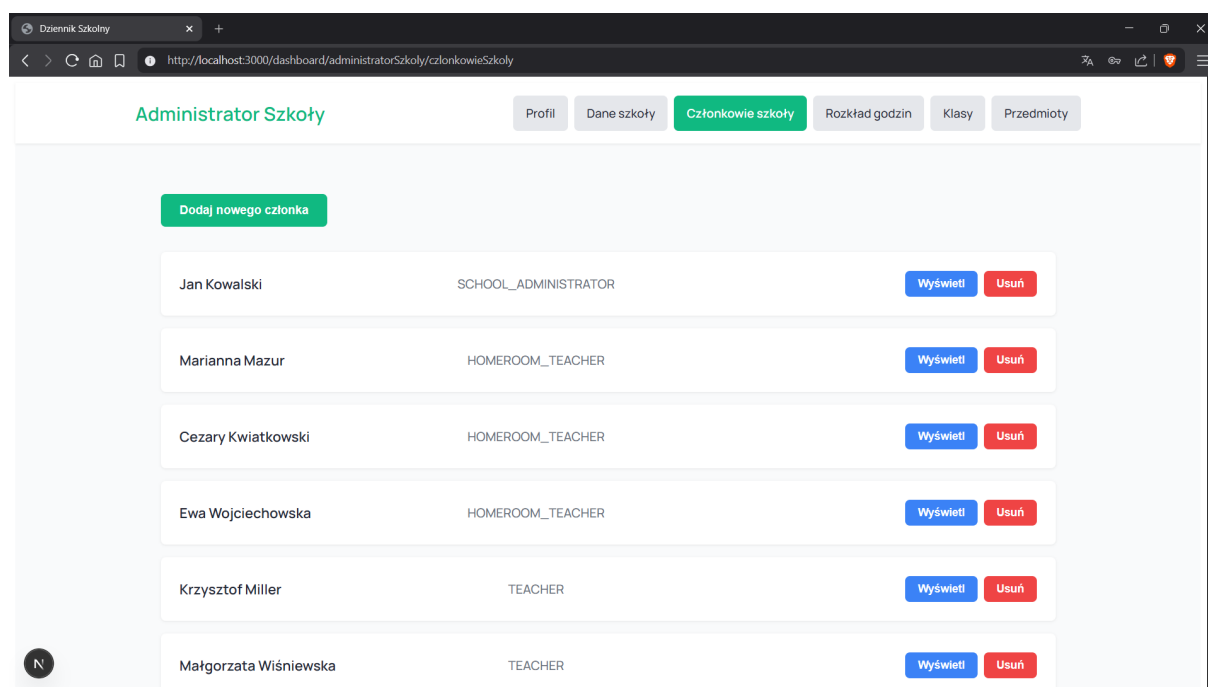
Administrator Aplikacji

[Profil](#) [Szkoły](#) [Wnioski](#)

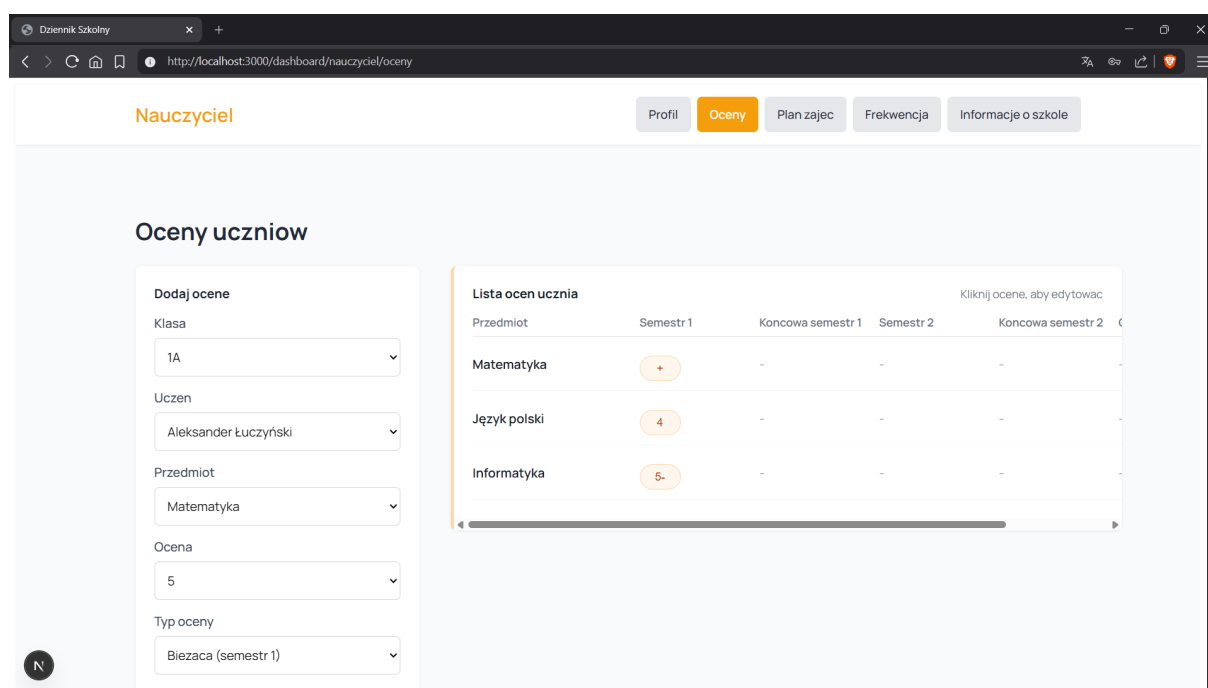
[Utwórz szkołę](#)

Szkoła Podstawowa nr 1 w Gdańsku	Aktywna
Data i godzina utworzenia 12.06.2026, 20:36	Data i godzina aktualizacji 12.06.2026, 20:36
Usuń	Wyświetl
Liceum Ogólnokształcące nr 1 w Gdańsku	Aktywna
Data i godzina utworzenia 12.06.2026, 20:36	Data i godzina aktualizacji 12.06.2026, 20:36
Usuń	Wyświetl
Szkoła Branżowa nr 1 w Gdańsku	Aktywna
Data i godzina utworzenia 12.06.2026, 20:36	Data i godzina aktualizacji 12.06.2026, 20:36
Usuń	Wyświetl

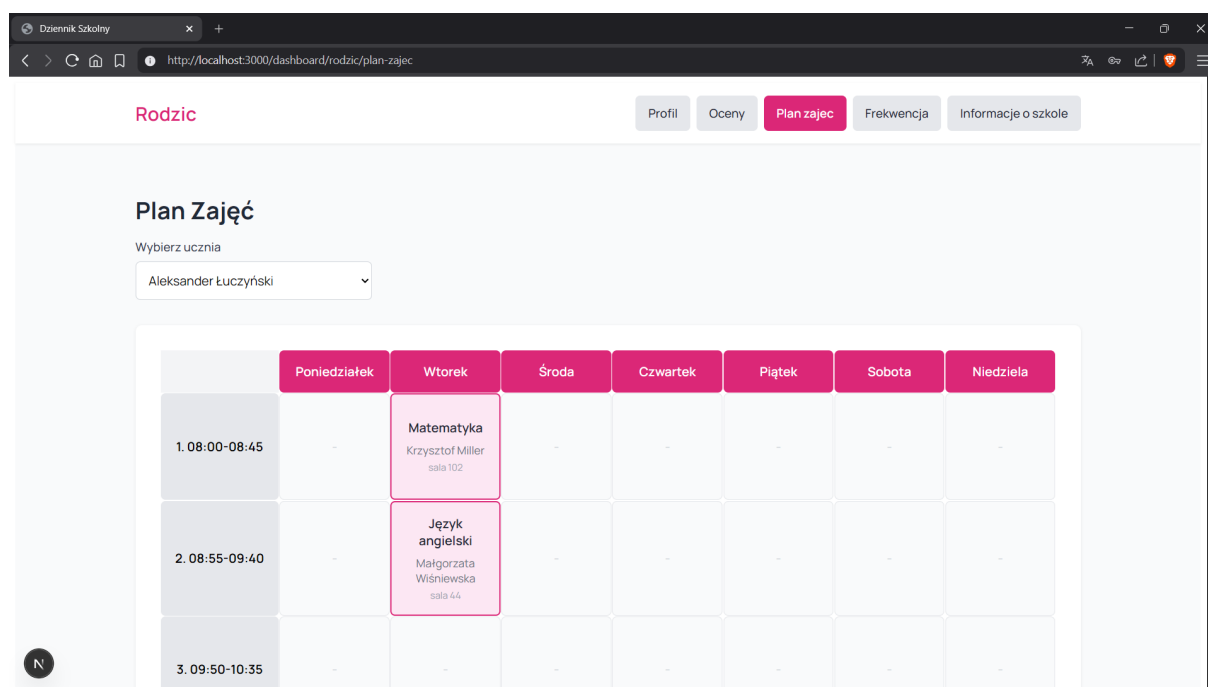
Rysunek 7: Strona panelu administratora aplikacji (lista szkół).



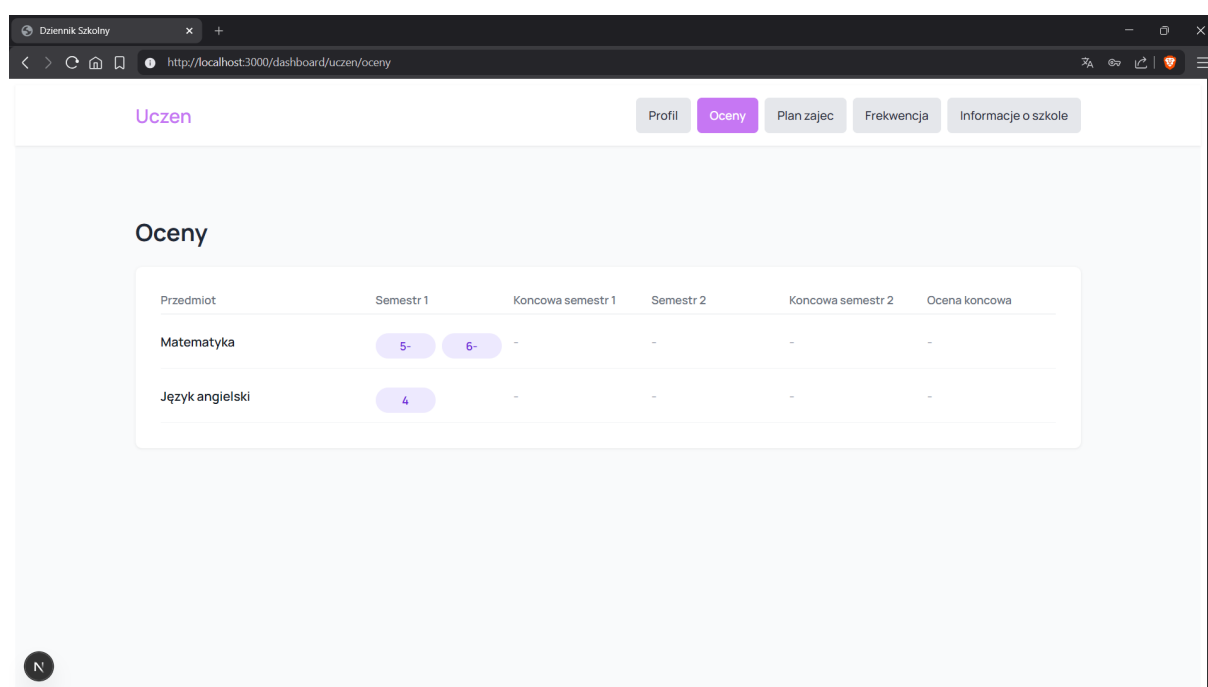
Rysunek 8: Strona panelu administratora szkoły (lista członków szkoły).



Rysunek 9: Strona panelu nauczyciela (sekcja wystawiania ocen).



Rysunek 10: Strona panelu rodzica (plan lekcji wybranego ucznia).



Rysunek 11: Strona panelu ucznia (lista ocen).

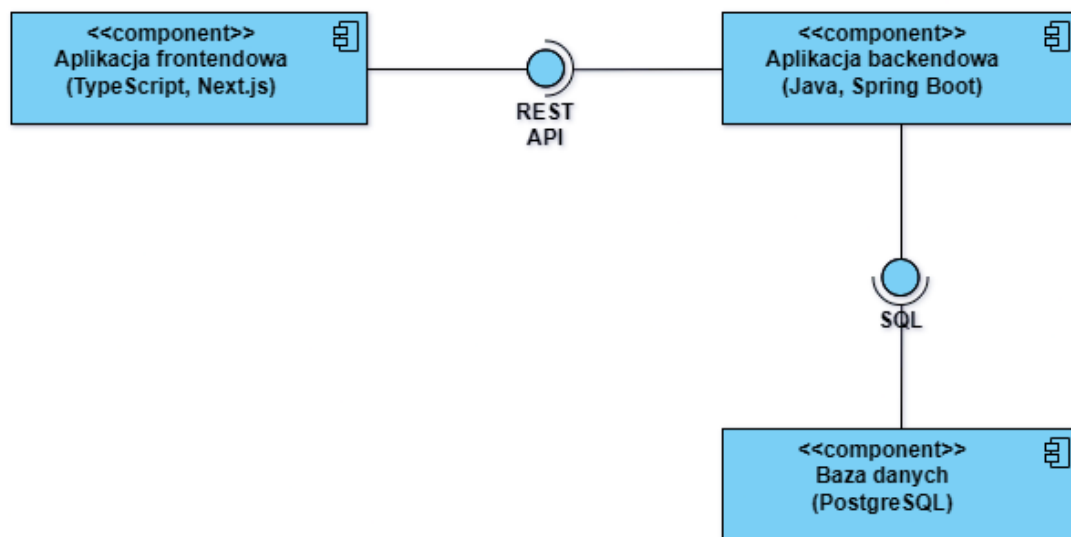
3 Implementacja

3.1 Architektura całości systemu

System dziennika elektronicznego został wykonany w architekturze monolitycznej i zawiera 3 główne komponenty:

- Aplikacja frontendowa
- Aplikacja backendowa
- Baza danych

Poniższy diagram przedstawia główną architekturę systemu.



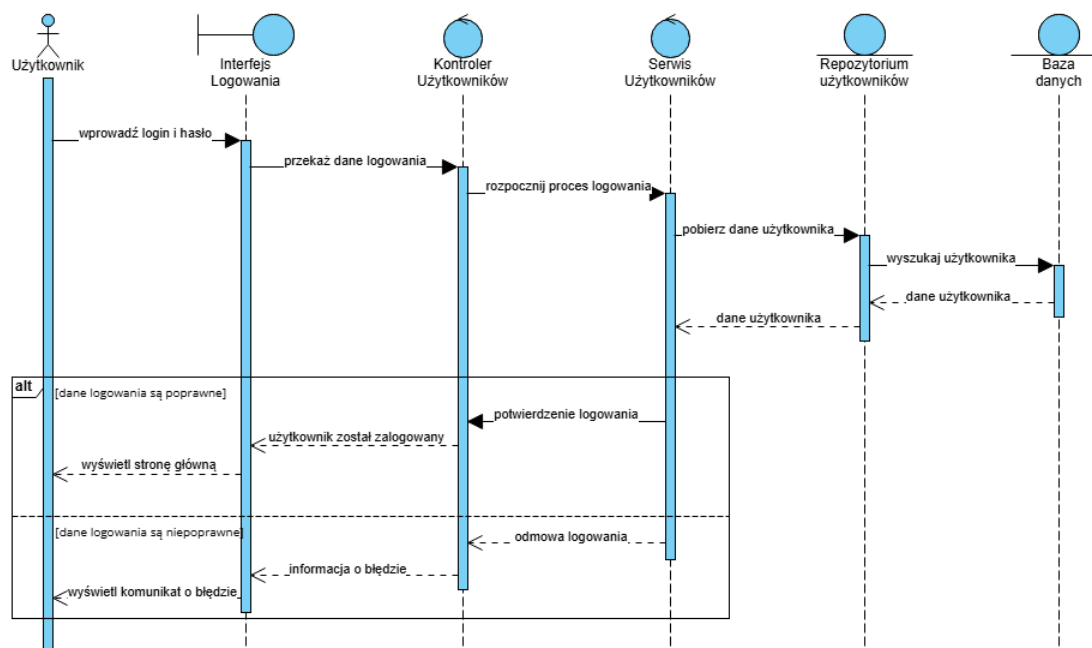
Rysunek 12: Diagram architektury systemu dziennika elektronicznego.

3.2 Użyte wzorce projektowe

System dziennika elektronicznego realizuje poniższe wzorce projektowe:

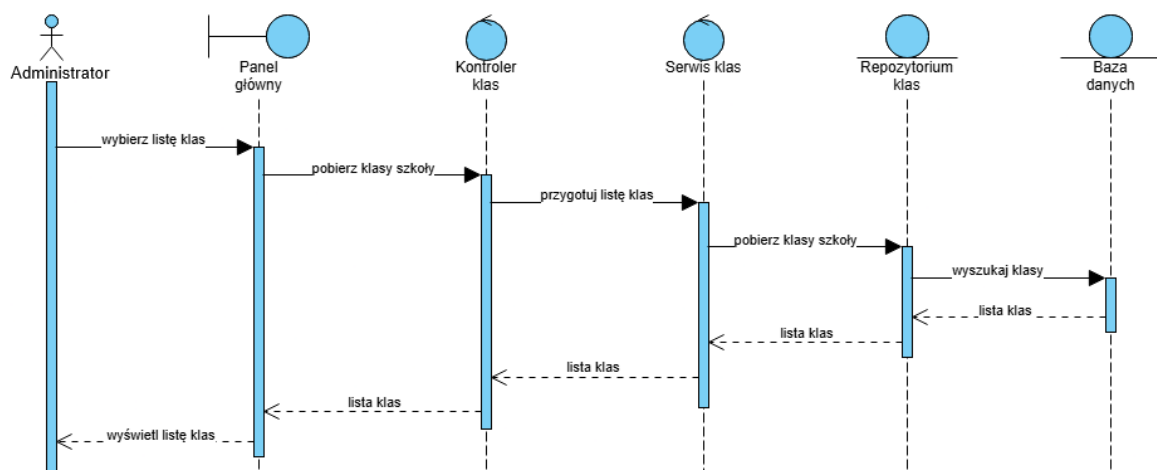
- **REST** (*Representational State Transfer*) - aplikacje frontendowa i backendowa komunikują się przez protokół HTTP oraz interfejs REST API wystawiany przez backend. Dane są wymieniane w formacie JSON. Dzięki temu wzorcowi, dane są w prosty sposób przesyłane pomiędzy aplikacjami.
- **ORM** (*Object-Relational Mapping*) - aplikacja backendowa używa mapowania obiektowo-relacyjnego, aby komunikować się z bazą danych. Dzięki temu wzorcowi, zapytania do bazy danych można formułować przy pomocy obiektów klas, a nie używając języka SQL.

3.3 Diagramy sekwencji przedstawiające funkcjonalności systemu



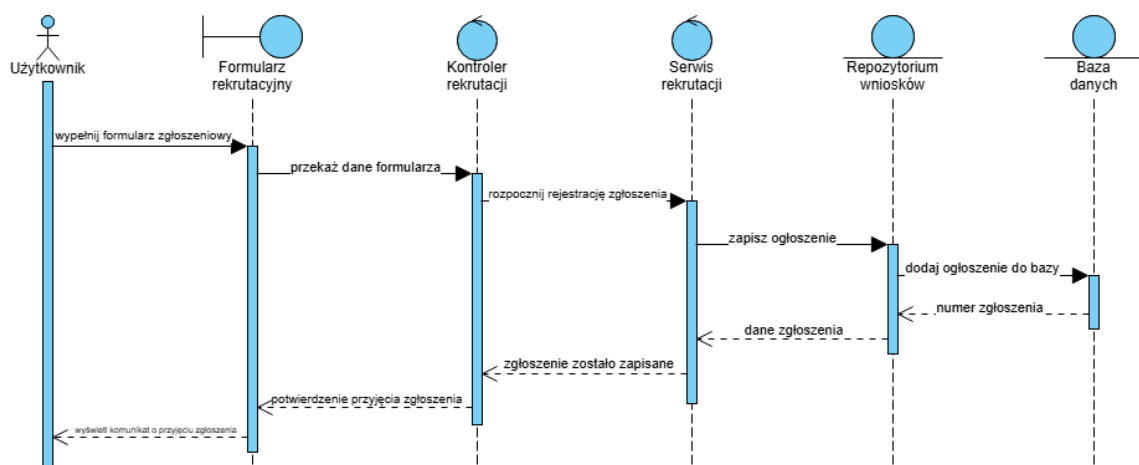
Rysunek 13: Logowanie do systemu

Powyższy diagram przedstawia logowanie się użytkownika do systemu dziennika elektronicznego. *Użytkownik* wprowadza dane logowania w *Interfejsie logowania*, które są przekazywane do warstwy *Kontrolera użytkowników* i *Serwisu użytkowników* odpowiedzialnego za obsługę logowania. Serwis pobiera dane użytkownika z *Repozytorium użytkowników*, które komunikuje się z *Bazą danych* w celu odnalezienia odpowiedniego konta. Po weryfikacji poprawności danych logowania system zwraca informację o pomyślnym logowaniu i wyświetla użytkownikowi stronę główną aplikacji. W przypadku podania niepoprawnych danych użytkownik otrzymuje komunikat o błędzie i nie ma dostępu do systemu.



Rysunek 14: Wyświetlanie listy klas

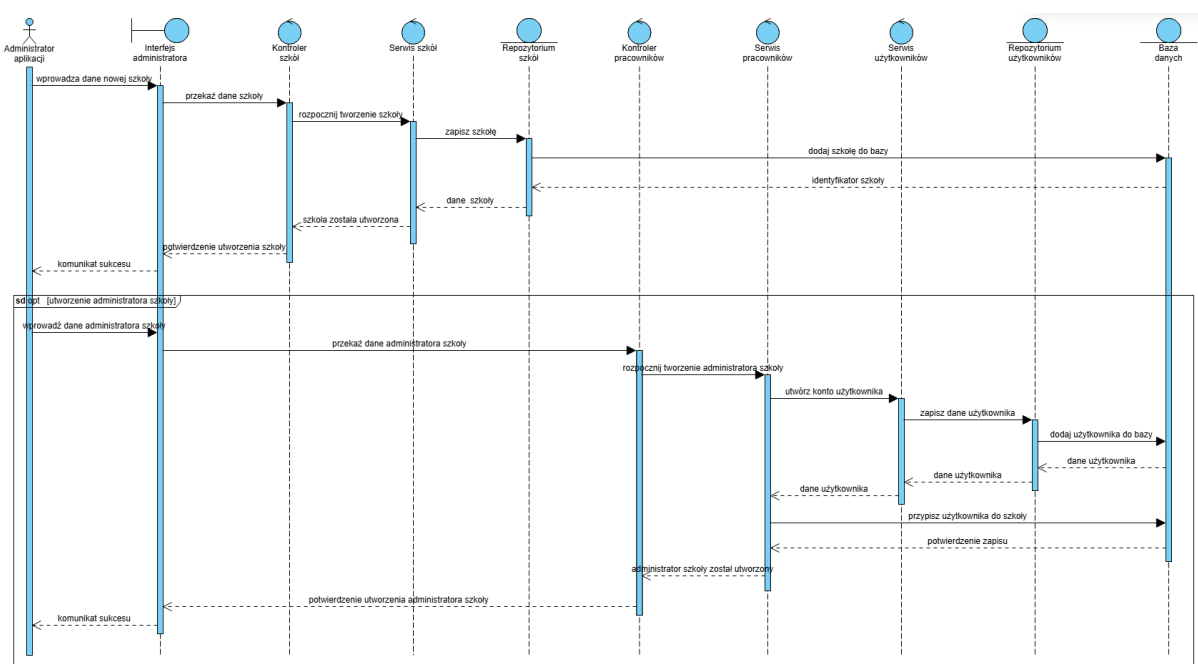
Powyższy diagram obrazuje proces pobierania i wyświetlania listy klas przypisanych do szkoły. **Administrator** wykonuje operację poprzez swój **Panel główny**. Jego żądanie jest przekierowane do **Kontrolera klas**, który następnie deleguje pobranie danych do **Serwisu klas**. Serwis komunikuje się z **Repozytorium klas** odpowiedzialnym za dostęp do danych zapisanych w **Bazie danych**. Po pobraniu listy klas informacje są zwracane do interfejsu użytkownika, gdzie są one wyświetlane w formie listy.



Rysunek 15: Składanie wniosku o utworzenie szkoły

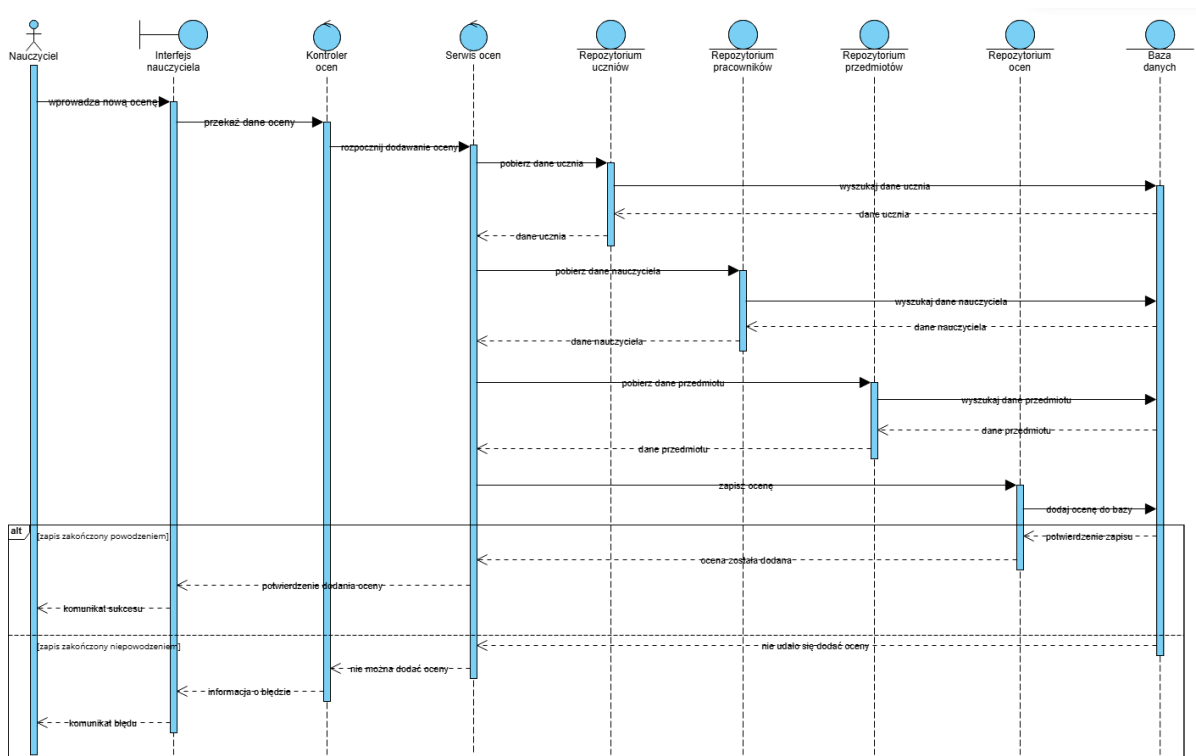
Powyższy diagram przedstawia proces składania wniosku o utworzenie szkoły przez **Użytkownika**. Wypełnia on **Formularz rekrutacyjny** i przesyła go do systemu. Tam dane formularza są przekazywane przez **Kontroler rekrutacji** do **Serwisu rekrutacji**, który obsługuje proces rejestracji zgłoszeń. Serwis zapisuje zgłoszenia przy użyciu **Repo-**

zytorium wniosków, które komunikuje się z *Bazą danych*. Po poprawnym zapisaniu zgłoszenia system generuje numer identyfikacyjny zgłoszenia i zwraca użytkownikowi potwierdzenie przyjęcia jego wniosku.



Rysunek 16: Utworzenie szkoły i przypisanie jej administratora

Powyższy diagram pokazuje sposób w jaki *Administrator aplikacji* dodaje nową szkołę do systemu oraz opcjonalnie przypisuje do niej administratora szkoły. Najpierw przy pomocy swojego *Interfejsu* wprowadza dane nowej placówki, które następnie są przekazywane do *Kontrolera szkół* i dalej do *Serwisu szkół*, który jest odpowiedzialny za zarządzanie szkołami. Następnie dane są zapisywane w *Bazie danych* za pośrednictwem *Repozytorium szkół*. Po pomyślnym utworzeniu szkoły administrator aplikacji otrzymuje komunikat o tym, że udało mu się zakończyć proces oraz otrzymuje on możliwość dodania do niej administratora szkoły. Dane nowego użytkownika są przekazywane do *Kontrolera pracowników*. Następnie poprzez *Serwis pracowników* i *Serwis użytkowników*, dane użytkownika zostają zapisane w bazie danych przez *Repozytorium użytkowników*. Po utworzeniu konta dla administratora zostaje on przypisany do wcześniej stworzonej szkoły. Po zakończeniu procesu system wyświetla administratorowi aplikacji komunikat o prawidłowym wykonaniu operacji.



Rysunek 17: Wpisywanie oceny

Powyższy diagram przedstawia proces wystawiania oceny przez *Nauczyciela*. Nauczyciel wprowadza dane oceny w *Interfejsie nauczyciela*, skąd zostają one przekazane do *Kontrolera ocen* oraz *Serwisu ocen*. W celu poprawnego utworzenia wpisu serwis pobiera informacje o: uczniu z *Repozytorium uczniów*, nauczycielu z *Repozytorium pracowników* i przedmiocie z *Repozytorium przedmiotów*. Po otrzymaniu wszystkich wymaganych danych serwis zapisuje ocenę poprzez *Repozytorium ocen*, które komunikuje się z *Bazą danych*. W przypadku poprawnego wykonania operacji nauczyciel otrzymuje potwierdzenie dodania oceny. Jeżeli podczas zapisu wystąpi jakiś błąd, system zwraca odpowiedni komunikat informujący o niepowodzeniu operacji.

3.4 Użyte technologie w projekcie systemu

Aplikacja frontendowa

- Użyty język programowania: TypeScript
- Użyty framework: Next.js
- Użyte biblioteki produkcyjne: React, React DOM, Next
- Użyte biblioteki deweloperskie: Tailwind CSS, ESLint, React Testing Library, Vitest

Aplikacja backendowa

- Użyty język programowania: Java
- Użyty framework: Spring Boot
- Użyte biblioteki produkcyjne: Spring Data JPA, Spring Web, PostgreSQL Driver
- Użyte biblioteki deweloperskie: Spring Data JPA Test, Spring Web Test, JUnit, JaCoCo, Lombok

4 Testy

4.1 Analiza testów części frontendowej aplikacji

4.1.1 Zakres testów

Testy frontendu aplikacji obejmowały weryfikację funkcjonalności poprzez:

- 39 testów jednostkowych (Vitest).
- 124 testy komponentów (Vitest + React Testing Library).
- Pokrycie kodu na poziomie ok. 83% (według v8 coverage).

Do pokrycia kodu wykorzystano narzędzie v8 coverage, które dostarcza szczegółowych informacji na temat wykonanych linii kodu, gałęzi warunkowych, metod i funkcji podczas testów automatycznych.

All files

82.92% Statements 2633/3175 **67.9% Branches** 1295/1907 **83.4% Functions** 608/729 **83.64% Lines** 2587/3093

Rysunek 18: Ogólne statystyki pokrycia kodu według v8 coverage

All files
83.26% Statements 431/496 65.2% Branches 288/339 85.71% Functions 36/385 85.14% Lines 487/478

Press n or j to go to the next uncovered block, b, p or k for the previous block.

Filter

File	Statements	Branches	Functions	Lines
administratorAplikacji	84.61% 11/13	50% 5/10	100% 3/3	84.61% 11/13
administratorAplikacjiKlasy	91.13% 72/79	60.46% 26/43	93.75% 15/16	94.52% 69/73
administratorAplikacjiProfil	79.68% 51/64	70% 28/40	70% 7/10	82.25% 51/62
administratorAplikacjiSzkoły	68% 34/50	57.14% 20/35	71.42% 10/14	70.21% 33/47
administratorAplikacjiSzkoły[id]	93.18% 41/44	81.57% 31/38	87.5% 7/8	97.61% 41/42
administratorAplikacjiSzkoły[id]utworz-administradora	86.48% 32/37	83.33% 20/24	87.5% 7/8	86.48% 32/37
administratorAplikacjiSzkoły/utworz	73.8% 31/42	76.66% 23/30	83.33% 5/6	73.17% 30/41
administratorAplikacjiUzytkownicy	83.72% 72/86	52% 26/50	90% 18/20	86.58% 71/82
administratorAplikacjiWnioski	85.71% 36/42	52.17% 12/23	100% 11/11	85.71% 36/42
administratorAplikacjiWnioski[id]	84.61% 33/39	65.38% 17/26	77.77% 7/9	84.61% 33/39

Rysunek 19: Pokrycie kodu dla modułu administratorAplikacji

All files

79.24% Statements 775/976 65.95% Branches 436/652 73.05% Functions 348/228 79.47% Lines 763/964

Press n or j to go to the next uncovered block, b, p or k for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
administratorSzkoly	84.61% 11/13	50% 5/10	100% 3/3	84.61% 11/13
administratorSzkoly/czlonkowieSzkoly	91.52% 54/59	69.23% 27/39	100% 15/15	92.59% 50/54
administratorSzkoly/czlonkowieSzkoly/[id]	68.12% 109/160	52.02% 77/148	57.14% 16/28	68.55% 109/159
administratorSzkoly/czlonkowieSzkoly/dodaj	87.5% 56/64	81.57% 31/38	88.23% 15/17	87.3% 55/63
administratorSzkoly/daneSzkoly	62.68% 42/67	58.33% 28/48	53.84% 7/13	63.63% 42/66
administratorSzkoly/klasy	100% 53/53	79.16% 19/24	100% 16/16	100% 51/51
administratorSzkoly/klasy/[id]	83.82% 57/68	69.04% 29/42	90% 9/10	83.58% 56/67
administratorSzkoly/klasy/[id]/plan-lekcji	75.15% 121/161	71.29% 77/108	55.76% 29/52	75% 120/160
administratorSzkoly/klasy/dodaj	83.01% 44/53	75% 24/32	88.88% 8/9	82.69% 43/52
administratorSzkoly/profil	79.68% 51/64	70% 28/40	70% 7/10	82.25% 51/62
administratorSzkoly/przedmioty	92.3% 36/39	68.75% 11/16	100% 10/10	91.89% 34/37
administratorSzkoly/przedmioty/[id]	80.95% 34/42	63.63% 14/22	83.33% 5/6	82.92% 34/41
administratorSzkoly/przedmioty/dodaj	80.64% 25/31	77.77% 14/18	80% 4/5	80.64% 25/31
administratorSzkoly/rozklad-godzin	78.84% 82/104	68.65% 46/67	64% 16/25	78.84% 82/104

Rysunek 20: Pokrycie kodu dla modułu administratorSzkoly

All files

83.9% Statements 376/442 74.32% Branches 159/212 81.17% Functions 93/95 84.72% Lines 366/432

Press n or j to go to the next uncovered block, b, p or k for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
nauczyciel	83.33% 10/12	50% 5/10	100% 3/3	83.33% 10/12
nauczyciel/frekwencja	78.26% 90/115	75.75% 50/66	84.21% 16/19	78.76% 89/113
nauczyciel/informacje-o-szkole	92.3% 24/26	75% 9/12	100% 3/3	92.3% 24/26
nauczyciel/oceny	85.05% 148/174	75% 72/96	77.5% 31/40	85.79% 145/169
nauczyciel/plan-zajec	94% 47/50	81.08% 30/37	90% 9/10	94% 47/50
nauczyciel/profil	79.68% 51/64	70% 28/40	70% 7/10	82.25% 51/62

Rysunek 21: Pokrycie kodu dla modułu nauczyciel

All files

86.11% Statements 845/788 71.67% Branches 332/464 87.41% Functions 132/151 86.85% Lines 842/778

Press n or j to go to the next uncovered block, b, p or k for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
rodzic	83.33% 10/12	50% 5/10	100% 3/3	83.33% 10/12
rodzic/frekwencja	94.18% 81/86	80.35% 45/56	95.23% 20/21	95.23% 80/84
rodzic/frekwencja/[id]	100% 26/26	83.33% 10/12	100% 3/3	100% 26/26
rodzic/informacje-o-szkole	82.6% 19/23	66.66% 8/12	100% 3/3	82.6% 19/23
rodzic/oceny	96.7% 88/91	81.35% 48/59	91.66% 22/24	96.62% 86/89
rodzic/oceny/[id]	87.5% 28/32	56.25% 9/16	100% 3/3	87.5% 28/32
rodzic/plan-zajec	71.79% 56/78	68.85% 42/61	68.42% 13/19	72.72% 56/77
rodzic/profil	80% 52/65	70% 28/40	70% 7/10	82.53% 52/63
uczen	83.33% 10/12	50% 5/10	100% 3/3	83.33% 10/12
uczen/frekwencja	92.42% 61/66	83.33% 30/36	93.33% 14/15	93.84% 61/65
uczen/frekwencja/[id]	80.76% 21/26	58.33% 7/12	100% 3/3	80.76% 21/26
uczen/informacje-o-szkole	61.53% 16/26	50% 6/12	100% 3/3	61.53% 16/26
uczen/oceny	87.5% 49/56	54.54% 18/33	87.5% 14/16	87.27% 48/55
uczen/oceny/[id]	93.75% 30/32	81.25% 13/16	100% 3/3	93.75% 30/32
uczen/plan-zajec	87.03% 47/54	78.04% 32/41	91.66% 11/12	87.03% 47/54
uczen/profil	79.68% 51/64	70% 28/40	70% 7/10	82.25% 51/62

Rysunek 22: Pokrycie kodu dla modułów rodzic i uczen

All files

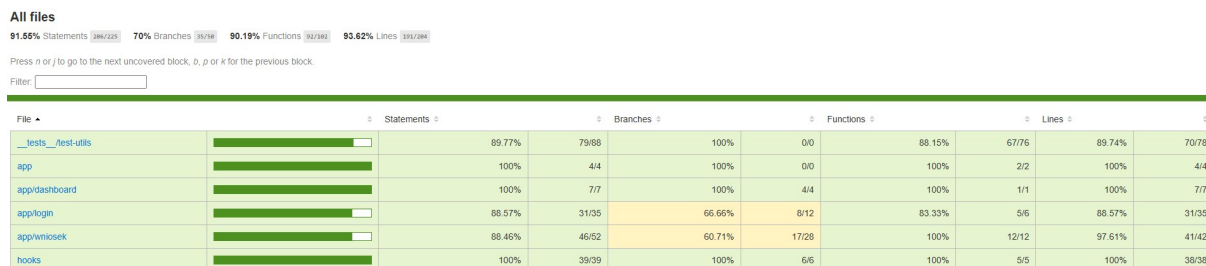
78.32% Statements 224/286 69.11% Branches 98/139 87.01% Functions 45/47 77.93% Lines 239/282

Press n or j to go to the next uncovered block, b, p or k for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
api.ts	76.15% 198/260	52.89% 73/138	96.72% 59/61	75.78% 194/256
auth.ts	100% 25/26	100% 21/21	100% 6/6	100% 25/25

Rysunek 23: Pokrycie kodu dla modułu lib



Rysunek 24: Pokrycie kodu dla pozostałych modułów

4.1.2 Kluczowe przypadki testowe

W procesie tworzenia testów wyznaczono poniższe przypadki testowe:

Testy jednostkowe

- **Wnioski i szkoły:** Weryfikacja budowania URL, obsługi statusów 201/204 oraz błędnych odpowiedzi (14 przypadków).
- **Operacje CRUD:** Testy operacji dla szkół, użytkowników, klas, uczniów, przedmiotów, lekcji, ocen i frekwencji (14 przypadków).
- **Autoryzacja:** Weryfikacja sesji, przekierowań zależnych od roli oraz uprawnień tras (6 przypadków).
- **Hook useProtectedRoute:** Testy przekierowań przy braku roli, błędach oraz poprawnym dostępie użytkownika (5 przypadków).

Testy komponentów

- **Layout i strony główne:** Renderowanie layoutu aplikacji, strony startowej, dashboardu oraz layoutów ról (9 przypadków).
- **Strony rodzica:** Testy profilu, szkoły, ocen, frekwencji oraz planu zajęć (5 przypadków).
- **Strony ucznia:** Testy profilu, szkoły, ocen, frekwencji oraz planu zajęć (6 przypadków).
- **Widoki szczegółowe:** Testy wyświetlania danych, obsługi błędów pobierania oraz fallbacków identyfikatorów (6 przypadków).
- **Frekwencja i oceny:** Testy błędów, braków danych, zmiany ucznia oraz renderowania warunkowego (11 przypadków).

- **Strony nauczyciela:** Testy frekwencji, ocen i planu zajęć z uwzględnieniem stanów ładowania i błędów (6 przypadków).
- **Administrator szkoły — klasy:** Testy listy klas, tworzenia, usuwania oraz pustego stanu (8 przypadków).
- **Administrator szkoły — szczegóły szkoły:** Testy dodawania administratora, komunikatów, nawigacji oraz renderowania danych (9 przypadków).
- **Administrator szkoły — członkowie:** Testy listy członków, filtrowania, dodawania, usuwania oraz obsługi błędów (11 przypadków).
- **Administrator aplikacji — klasy i użytkownicy:** Testy zarządzania klasami, list uczniów, edycji oraz stanów ładowania (11 przypadków).
- **Administrator aplikacji — wnioski:** Testy listy wniosków, akceptacji, odrzucania oraz podglądu szczegółów (5 przypadków).
- **Logowanie:** Testy procesu logowania, zapisu sesji oraz obsługi błędów (3 przypadki).
- **Przedmioty:** Testy listy przedmiotów, dodawania oraz edycji (5 przypadków).
- **Plan lekcji:** Testy dodawania, usuwania oraz wyświetlania siatki zajęć (4 przypadki).
- **Rozkład godzin:** Testy renderowania godzin oraz dodawania i usuwania wpisów (5 przypadków).

4.1.3 Podsumowanie wyników testów

Moduł	Liczba testów	Stan
Wnioski i szkoły	14	100% sukces
Operacje CRUD	14	100% sukces
Autoryzacja	6	100% sukces
Hook useProtectedRoute	5	100% sukces

Tabela 1: Statystyki testów jednostkowych

Moduł	Liczba testów	Stan
Layout i strony główne	9	100% sukces
Strony rodzica	5	100% sukces
Strony ucznia	6	100% sukces
Widoki szczegółowe	6	100% sukces
Frekwencja i oceny	11	100% sukces
Strony nauczyciela	6	100% sukces
Administrator szkoły — klasy	8	100% sukces
Administrator szkoły — szczegóły szkoły	9	100% sukces
Administrator szkoły — członkowie	11	100% sukces
Administrator aplikacji — klasy i użytkownicy	11	100% sukces
Administrator aplikacji — wnioski	5	100% sukces
Logowanie	3	100% sukces
Przedmioty	5	100% sukces
Plan lekcji	4	100% sukces
Rozkład godzin	5	100% sukces

Tabela 2: Statystyki testów komponentów

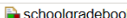
4.2 Analiza testów części backendowej aplikacji

4.2.1 Zakres testów

Testy backendu aplikacji obejmowały weryfikację funkcjonalności poprzez:

- 128 testów jednostkowych (JUnit + Mockito).
- Pokrycie kodu na poziomie ok. 73 % (według JaCoCo).

Do sprawdzenia pokrycia kodu testami użyto narzędzia JaCoCo (Java Code Coverage), które przedstawia szczegółowe informacje o wykonanych liniach kodu, metodach, klasach oraz rozgałęzieniach warunkowych.



schoolgradebook

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
pl.edu.ug.schoolgradebook.controller		0%		0%	71	71	194	194	61	61	11	11
pl.edu.ug.schoolgradebook.domain		0%		0%	10	10	28	28	9	9	5	5
pl.edu.ug.schoolgradebook.api		0%		n/a	7	7	20	20	7	7	1	1
pl.edu.ug.schoolgradebook		0%		n/a	2	2	3	3	2	2	1	1
pl.edu.ug.schoolgradebook.service		100%		90%	6	113	0	367	0	80	0	12
pl.edu.ug.schoolgradebook.util.mapper		100%		100%	0	37	0	194	0	36	0	11
pl.edu.ug.schoolgradebook.enums		100%		n/a	0	6	0	53	0	6	0	6
pl.edu.ug.schoolgradebook.exception		100%		n/a	0	4	0	8	0	4	0	4
Total	927 of 3 467	73%	28 of 90	68%	96	250	245	867	79	205	18	51

Rysunek 25: Pokrycie kodu dla całej aplikacji backendowej

schoolgradebook > pl.edu.ug.schoolgradebook.service

pl.edu.ug.schoolgradebook.service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
StudentService		100%		83%	3	16	0	50	0	7	0	1
LessonService		100%		83%	1	10	0	43	0	7	0	1
UserService		100%		93%	1	21	0	50	0	13	0	1
GradeService		100%		n/a	0	6	0	33	0	6	0	1
SchoolClassService		100%		87%	1	10	0	33	0	6	0	1
SchoolMemberService		100%		100%	0	10	0	37	0	7	0	1
LessonTimeService		100%		100%	0	11	0	29	0	7	0	1
AttendanceService		100%		100%	0	7	0	27	0	6	0	1
SchoolService		100%		n/a	0	8	0	28	0	8	0	1
SubjectService		100%		100%	0	6	0	18	0	5	0	1
SchoolApplicationService		100%		n/a	0	5	0	15	0	5	0	1
EntityService		100%		n/a	0	3	0	4	0	3	0	1
Total	0 of 1 735	100%	6 of 66	90%	6	113	0	367	0	80	0	12

Rysunek 26: Pokrycie kodu dla modułu klas serwisowych

schoolgradebook > pl.edu.ug.schoolgradebook.util.mapper

pl.edu.ug.schoolgradebook.util.mapper

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
SchoolApplicationMapper		100%		n/a	0	4	0	30	0	4	0	1
SchoolMapper		100%		n/a	0	4	0	28	0	4	0	1
GradeMapper		100%		n/a	0	3	0	23	0	3	0	1
StudentMapper		100%		100%	0	4	0	15	0	3	0	1
UserMapper		100%		n/a	0	4	0	19	0	4	0	1
LessonMapper		100%		n/a	0	3	0	17	0	3	0	1
AttendanceMapper		100%		n/a	0	3	0	17	0	3	0	1
SchoolMemberMapper		100%		n/a	0	3	0	14	0	3	0	1
LessonTimeMapper		100%		n/a	0	3	0	11	0	3	0	1
SchoolClassMapper		100%		n/a	0	3	0	11	0	3	0	1
SubjectMapper		100%		n/a	0	3	0	9	0	3	0	1
Total	0 of 486	100%	0 of 2	100%	0	37	0	194	0	36	0	11

Rysunek 27: Pokrycie kodu dla modułu klas mapujących

4.2.2 Kluczowe przypadki testowe

W procesie tworzenia testów jednostkowych wyznaczono poniższe przypadki testowe:

- **Zarządzanie wnioskami o rejestrację szkoły:** Testy tworzenia, wyszukiwania, edycji i usuwania wniosków (7 przypadków).
- **Zarządzanie szkołami:** Testy tworzenia, wyszukiwania, edycji i usuwania szkół (11 przypadków).
- **Zarządzanie użytkownikami:** Rejestracja, logowanie, edycja, usuwanie, podział na role użytkowników oraz weryfikacja sytuacji kolizyjnych (38 przypadków).
- **Zarządzanie lekcjami:** Tworzenie, wyszukiwanie, edycja, usuwanie lekcji oraz godzin lekcyjnych oraz weryfikacja możliwych kolizji w planach zajęć (18 przypadków).
- **Zarządzanie klasami:** Tworzenie, wyszukiwanie, edycja, usuwanie klas oraz przypisywanie do nich użytkowników (9 przypadków).

- **Zarządzanie przedmiotami szkolnymi:** Tworzenie, wyszukiwanie, edycja, usuwanie przedmiotów i weryfikacja powtórzeń przedmiotów dla szkoły (6 przypadków).
- **Zarządzanie ocenami:** Dodawanie, wyszukiwanie, edycja i usuwanie ocen (6 przypadków).
- **Zarządzanie frekwencją:** Dodawanie, wyszukiwanie, edycja i usuwanie frekwencji (7 przypadków).
- **Mapowanie danych:** Mapowanie obiektów DTO (Data Transfer Object) oraz encji JPA (Java Persistence API) (26 przypadków).

4.2.3 Podsumowanie wyników testów

Moduł	Liczba testów	Stan
Zarządzanie wnioskami	7	100% sukces
Zarządzanie szkołami	11	100% sukces
Zarządzanie użytkownikami	38	100% sukces
Zarządzanie lekcjami	18	100% sukces
Zarządzanie klasami	9	100% sukces
Zarządzanie przedmiotami	6	100% sukces
Zarządzanie ocenami	6	100% sukces
Zarządzanie frekwencją	7	100% sukces
Mapowanie obiektów	26	100 % sukces

Tabela 3: Statystyki testów jednostkowych

4.3 Wnioski z testów

Wszystkie 166 testów po stronie frontendowej zakończyło się powodzeniem, a aplikacja poprawnie obsługuje przypadki brzegowe takie jak brakujące dane, błędy API czy brak uprawnień użytkownika. Dzięki testom jednostkowym kluczowych modułów pomocniczych (api.ts, auth.ts) można było szybko weryfikować poprawność komunikacji z serwerem oraz zarządzania sesją użytkownika. Testy komponentów pozwoliły na weryfikację interakcji między UI, routinguem i logiką biznesową. Osiągnięte pokrycie kodu wynosi 83% co można uznać za zadowalający wynik.

Wszystkie 128 testów po stronie backendowej zakończyło się powodzeniem, a aplikacja poprawnie zarządza danymi pomiędzy backendem a bazą danych. Testy jednostkowe warstwy serwisowej czy metod mapujących pozwoliły zweryfikować, czy dane są poprawnie mapowane oraz przechowywane w bazie danych. Przetestowano także przypadki konfliktowe pomiędzy danymi np. próba rejestracji na ten sam login czy próba niepoprawnego ustanowienia godzin lekcyjnych. Osiągnięte pokrycie kodu na poziomie 73% można uznać za dobry wynik.